# OFFLINE NOTEBOOK DOCUMENT QA

## A Secure, Persistent and Multimodal Retrieval-Augmented Generation System

*Project Report for Guide Review*

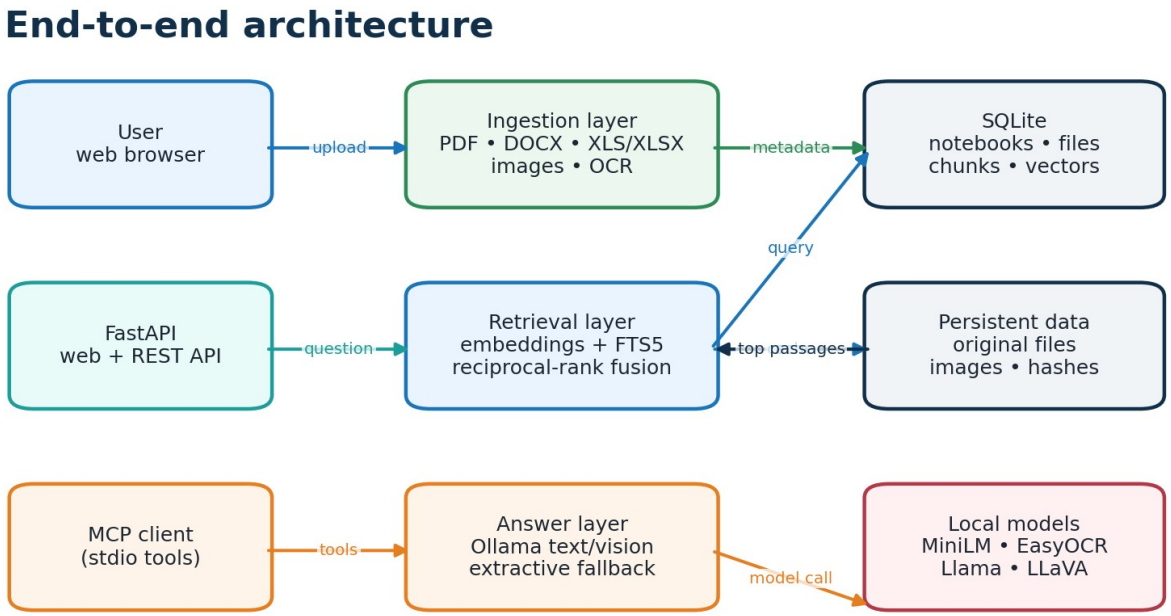



*Figure 1. System overview*

Prepared by: Cyril P. George
Repository: github.com/dhxvxn/URSC_RAG
Date: 10 November 2026

# Abstract and executive summary

*What the system delivers*

This project implements an offline document question-answering service for secure offices and laboratories. Users create notebooks, add files or folders, and ask questions against one selected collection. The application extracts text from common office formats, applies OCR to scanned pages and images, creates local sentence embeddings, and stores the original files, images, chunks and vectors persistently. Later uploads of the same bytes are recognized by SHA-256 and reused without repeated embedding work.

Retrieval combines dense semantic similarity with SQLite FTS5 lexical search. This hybrid path is designed to retrieve both conceptually related passages and exact terms such as identifiers, acronyms and mission codes. Ollama supplies local text or vision answers using only retrieved evidence. If Ollama is unavailable, the service returns an extractive answer containing the strongest indexed passages.

| Capability | Implemented behavior | Value to the lab |
|---|---|---|
| Persistent indexing | SQLite catalog, stored documents, embeddings and images | No repeated uploads or re-indexing |
| Multimodal retrieval | Text chunks, OCR text and image sources | Questions can use reports, scans and figures |
| Offline operation | Native Python/Ollama or Docker after preparation | Suitable for isolated networks |
| Notebook scope | Queries restricted to selected notebook | Separate topics and projects |
| Integration | FastAPI, Swagger and MCP tools | Browser, API and agent workflows |

The report also identifies the next design step: a first-class topic and entity knowledge graph. The current implementation persists notebook–document–chunk relationships; the graph extension should add extracted entities, topics, relations and graph-aware ranking while preserving the existing local retrieval path.

# Problem statement and motivation

*Why an offline RAG system is needed*

Secure offices often contain large collections of mission reports, technical notes, spreadsheets, scanned documents and diagrams. Manual search is slow because information is distributed across file types and folders. Cloud AI services may be unsuitable when documents are sensitive, when internet access is unavailable, or when the organization requires data to remain inside the lab.

The system addresses this gap by moving ingestion, indexing, retrieval and answer generation onto one controlled computer. A prepared machine may download software and model weights once. After that, the runtime can be disconnected from the internet while users continue to add documents and ask questions over the local store.

| Challenge | Conventional workflow | Project response |
|---|---|---|
| Repeated uploads | User repeatedly attaches the same files | Content hash and persistent document catalog |
| Mixed formats | Separate viewers and search tools | PDF, Word, Excel and image extractors |
| Scanned material | Text search misses image-only pages | EasyOCR and image retention |
| Exact terminology | Embeddings may underweight codes | FTS5 lexical signal fused with vectors |
| Restricted network | Cloud APIs are unreachable or prohibited | Local Ollama and local storage |
| Topic separation | A single shared search pool creates noise | Notebook-scoped retrieval |

The intended first deployment is a single Ubuntu or Windows PC for demonstration. Once the workflow is accepted, the same API can be bound to a protected lab interface so multiple workstations use the server without receiving the document store or Ollama port.

# Objectives, scope and requirements

*Functional and operational targets*

The primary objective is to provide fast, repeatable question answering over a persistent local knowledge base. The design balances retrieval speed, evidence traceability, offline operation and the ability to add material incrementally.

| Requirement | Acceptance interpretation |
|---|---|
| Persistent files | Original uploads and extracted images survive restarts and rebuilds. |
| One-time embeddings | Same SHA-256 bytes return cached and are not embedded again. |
| Incremental folders | A later folder scan indexes only new or changed files. |
| Supported formats | PDF, DOCX, XLS/XLSX and common raster images. |
| Fast retrieval | Notebook matrix cached in memory; semantic and lexical search fused. |
| Image support | Image OCR participates in retrieval; vision model can describe sources. |
| Offline runtime | No cloud API is required after preparation. |
| Network deployment | API can be exposed to a lab subnet while Ollama remains private. |
| Integration | REST, Swagger and MCP interfaces share the same database. |

Out of scope for the current baseline are enterprise identity management, a production audit trail, distributed vector databases and automatic graph extraction. These are documented as controlled next steps rather than treated as completed functionality.

# System architecture

*Components and data flow*

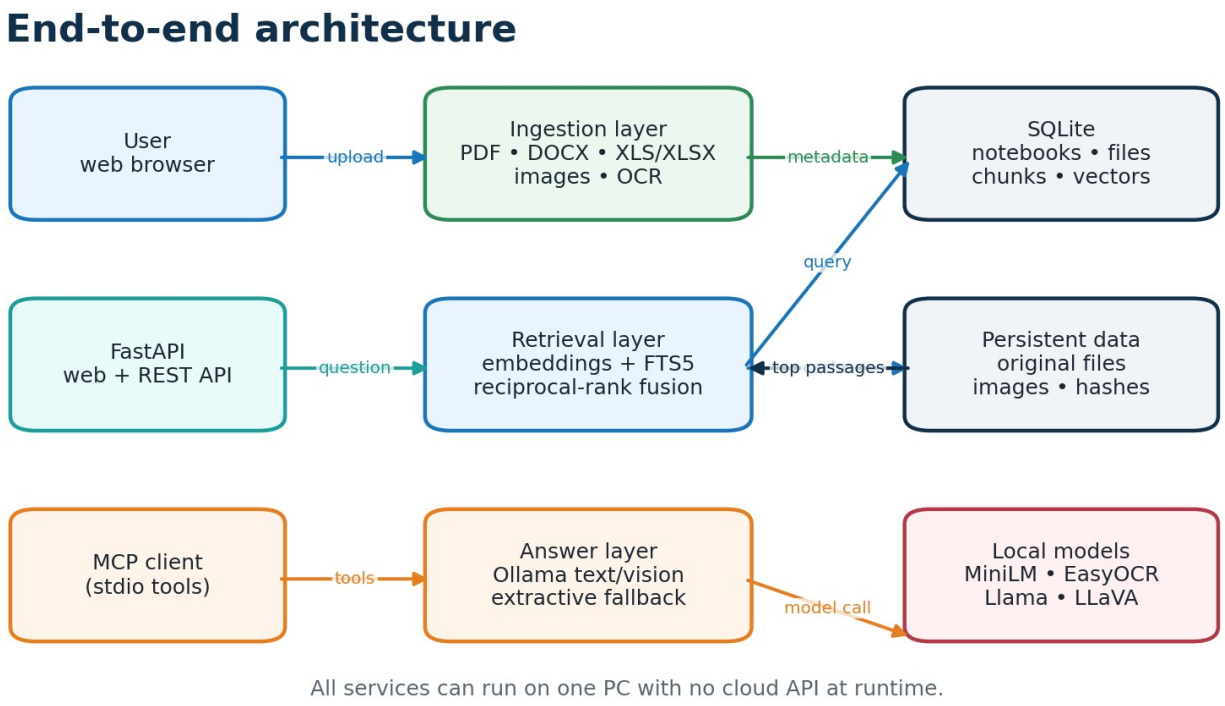



*Figure 2. Components communicate locally; document evidence stays on the host.*

FastAPI is the application boundary. The web interface is served by the same process, and the REST endpoints are available through the interactive Swagger documentation at /docs. MCP exposes notebook operations to compatible tool clients through stdio. Both interfaces call the same ingestion and retrieval functions, so deduplication and notebook scope are consistent.

SQLite provides the catalog and the durable embedding store. The application also retains the original file and extracted image paths. Ollama is treated as a local inference dependency: the API sends only bounded context and selected image payloads to the loopback service.

# Multiformat ingestion

*From files and folders to indexed evidence*

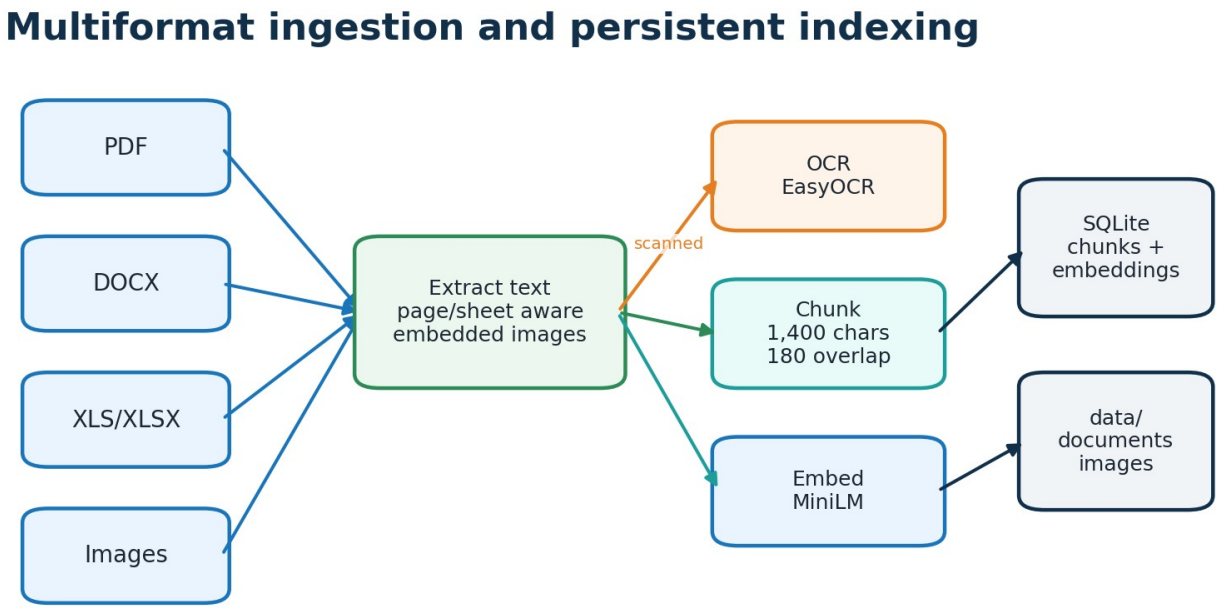



*Figure 3. Ingestion pipeline for text, tables, scans and images.*

The upload endpoint accepts multiple files in one request, and the browser file input supports selecting an entire folder. The MCP ingest_folder tool recursively scans a folder and passes each supported file through the same ingest_document routine. Unsupported extensions are reported as skipped rather than silently discarded.

PDF text is extracted page by page. DOCX paragraphs and tables are read from the package structure. Excel workbooks are converted to a readable representation for each sheet. Embedded media is extracted from PDF, DOCX and XLSX packages, retained under the data/images directory, and associated with its source document.

| Input | Extraction result | Stored evidence |
|---|---|---|
| PDF | Page text; rendered-page OCR when text is absent | Original PDF, chunks, images |
| DOCX | Paragraphs and table rows | Original DOCX, chunks, embedded images |
| XLS/XLSX | Sheet name plus tabular text | Original workbook, chunks, media |
| Raster image | OCR text where available | Original image, OCR text, image URL |

# OCR and image evidence

*Making scanned material searchable*

Office archives frequently contain scanned pages, diagrams and photographs. When a PDF page has no selectable text, PyMuPDF renders the page and EasyOCR attempts to extract text. Embedded images receive their own OCR pass. OCR text is appended to the relevant page or image context so it can participate in embedding and lexical retrieval.

Images are not reduced to text only. The original extracted image is preserved and returned as a source URL. When a query retrieves a document with images, the service can pass a bounded number of images to the local LLaVA vision model. This allows answers to refer to chart labels, diagram structure or other visual details when the model is installed.

| Evidence path | Benefit | Limitation |
| --- | --- | --- |
| Native PDF text | Fast and precise for digital documents | Unavailable for image-only pages |
| Page OCR | Recovers searchable text from scans | Quality depends on resolution and layout |
| Image OCR | Finds labels and captions inside figures | May miss handwriting or complex diagrams |
| Vision model | Adds visual interpretation | Requires more RAM and model storage |

For a controlled deployment, OCR models should be downloaded during preparation and copied with the offline bundle. At runtime, EasyOCR is configured with download_enabled=False, preventing accidental internet access.

# Embedding and indexing

*Turning documents into reusable vectors*

After extraction, text is normalized and divided into chunks of approximately 1,400 characters with 180 characters of overlap. Chunking prevents very long documents from overwhelming the context window while keeping enough local continuity for technical explanations. Each chunk is encoded by the all-MiniLM-L6-v2 sentence-transformer model and stored as a normalized float32 vector.

The vector is stored as a SQLite BLOB alongside document ID, page number and chunk text. At query time, the selected notebook's rows are loaded into an in-memory matrix. Because vectors are normalized, a matrix multiplication provides cosine similarity without a remote vector service.

| Index field | Purpose |
|---|---|
| document_id | Links chunks to original file and notebook membership |
| page | Preserves page or sheet provenance |
| text | Evidence supplied to lexical search and Ollama |
| embedding | Dense semantic representation |
| embedding_dim | Records vector shape for safe decoding |
| chunk_fts | SQLite FTS5 mirror for exact term retrieval |

The embedding model is loaded once at application startup when PRELOAD_EMBEDDING_MODEL is enabled. This removes the model-load penalty from the first user query and is a direct optimization for demonstrations and repeated use.

# Hybrid retrieval and speed

*Combining meaning with exact terms*

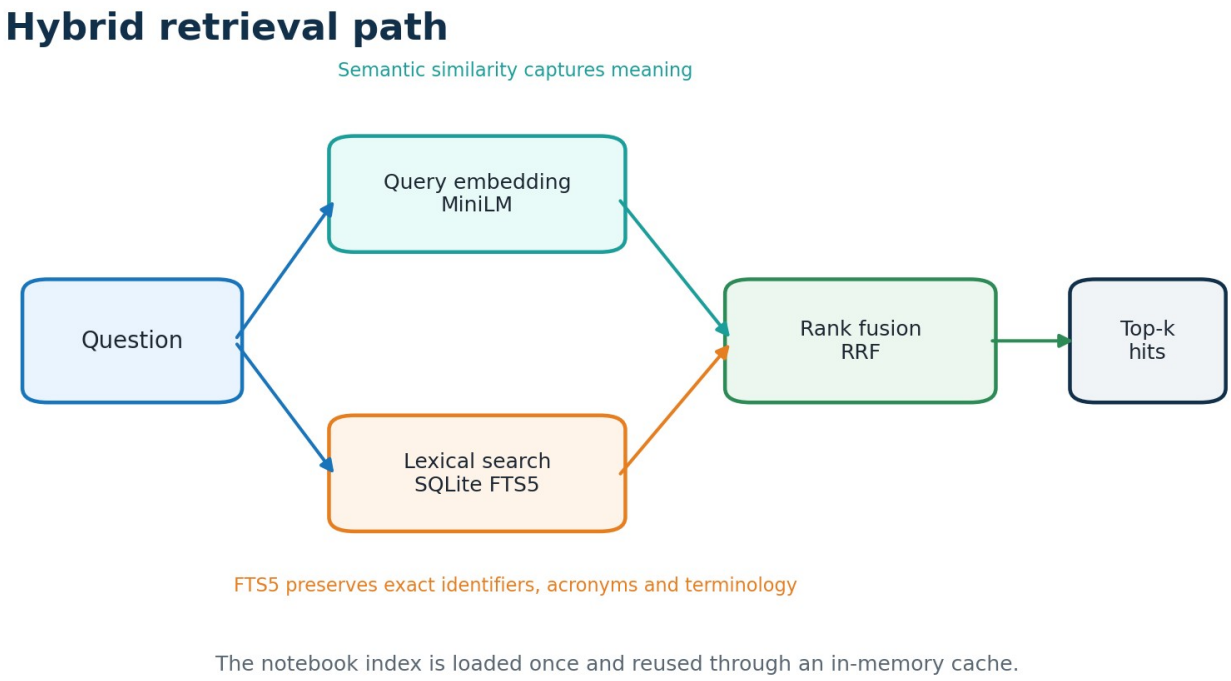



*Figure 4. Retrieval uses complementary semantic and lexical signals.*

Dense retrieval is effective when the question and source use different wording. Lexical retrieval is valuable for exact identifiers, acronyms, filenames and scientific notation. The service retrieves candidates from both paths and applies reciprocal-rank fusion (RRF): each result contributes 1/(60 + rank), and the combined score determines the top-k passages.

Two implementation choices reduce latency. First, the notebook embedding matrix is cached in memory and invalidated only after ingestion. Second, Ollama receives only the selected passages rather than the entire database. The API response exposes retrieval and answer timings separately, making it possible to demonstrate that model generation is usually the longer phase.

| Optimization | Effect |
|---|---|
| In-memory notebook matrix | Avoids repeated SQLite BLOB decoding for every question |
| FTS5 candidate search | Fast exact-term filtering using SQLite's native index |
| RRF fusion | Combines signals without collection-specific score calibration |
| Top-k and context limits | Bounds prompt size and model latency |
| Keep-alive | Keeps the Ollama model loaded between questions |

# Answer generation and image-aware QA

*Evidence-grounded local responses*

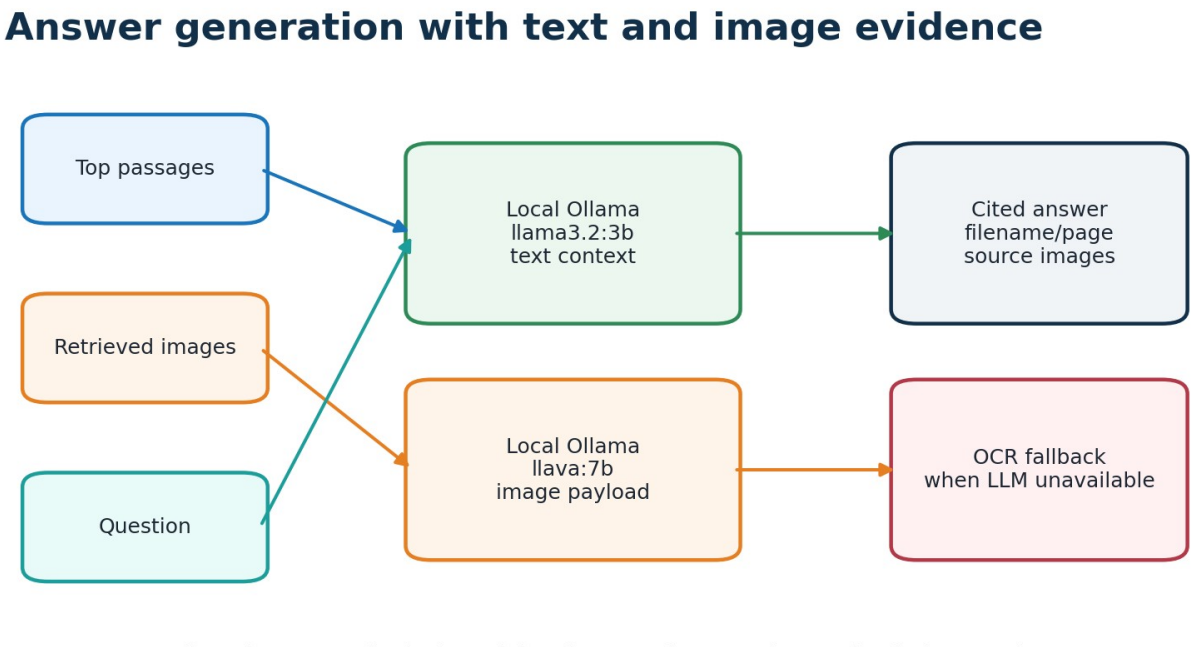



*Figure 5. Text and visual evidence share the same local answer path.*

The answer prompt instructs the local model to use only supplied notebook sources, cite filename and page where available, and state when the evidence is insufficient. The text model is llama3.2:3b by default. If images are attached to the retrieved sources, the application selects llava:7b as the vision model and includes up to MAX_VISION_IMAGES images.

If the Ollama request fails, the service does not return an empty response. It uses extractive_fallback to list the strongest indexed passages with source filenames and page numbers. This preserves useful search behavior during model startup, low-memory conditions or a temporarily stopped Ollama process.

| Mode | Default model | Use case |
|---|---|---|
| Text answer | llama3.2:3b | Technical questions over retrieved passages |
| Vision answer | llava:7b | Questions involving retrieved diagrams/images |
| Fallback | None | Evidence display when Ollama is unavailable |

# Notebook model and information organization

*Topic-scoped collections*

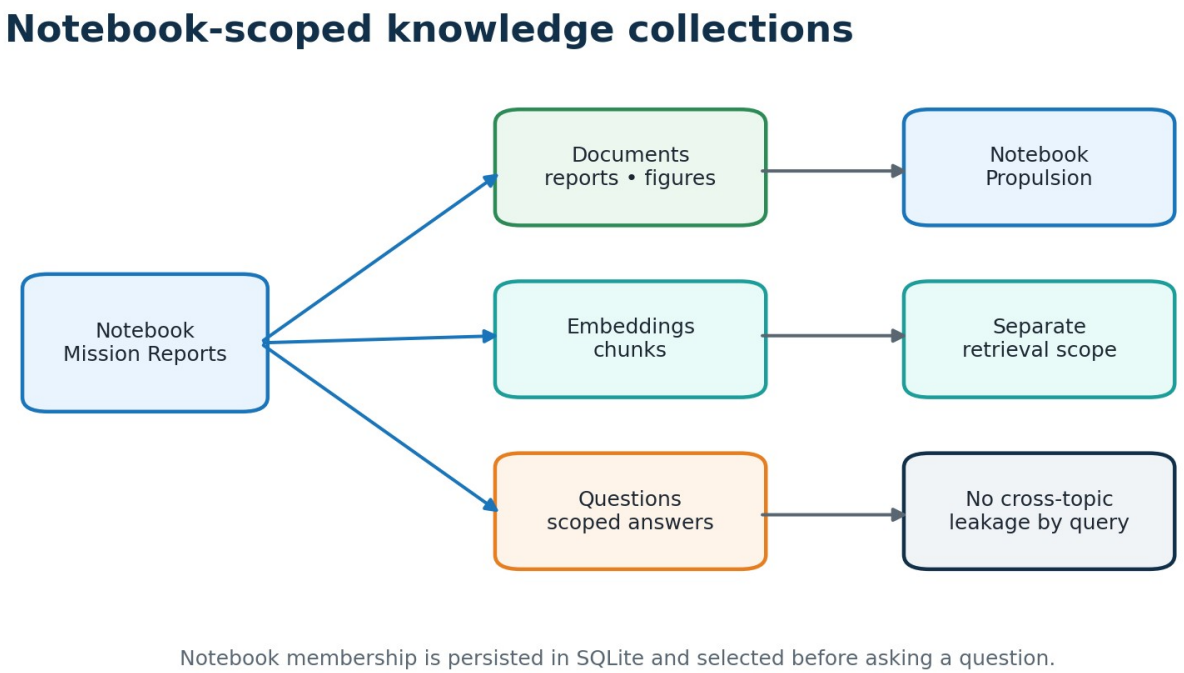



*Figure 6. Notebook membership limits retrieval to the selected topic.*

A notebook is a named collection such as Mission Reports, Propulsion or Finance. Documents are linked to notebooks through a join table rather than copied, so one file can belong to multiple collections without duplicate storage. Questions always include a notebook ID, and retrieval joins chunks through notebook membership before ranking.

This scope is important in a lab with multiple projects. It reduces unrelated hits, makes source lists easier to review and establishes the boundary needed for future access control. The browser interface exposes notebook creation, file selection, indexing and question asking in one flow.

| User action | Persistent result |
|---|---|
| Create notebook | Unique notebook row with creation time |
| Select files/folder | Membership links plus indexed document records |
| Ask question | Top passages and images from only that notebook |
| Add files later | Existing bytes cached; new/changed bytes indexed |

# Deduplication and incremental updates

*Why documents do not need to be uploaded again*

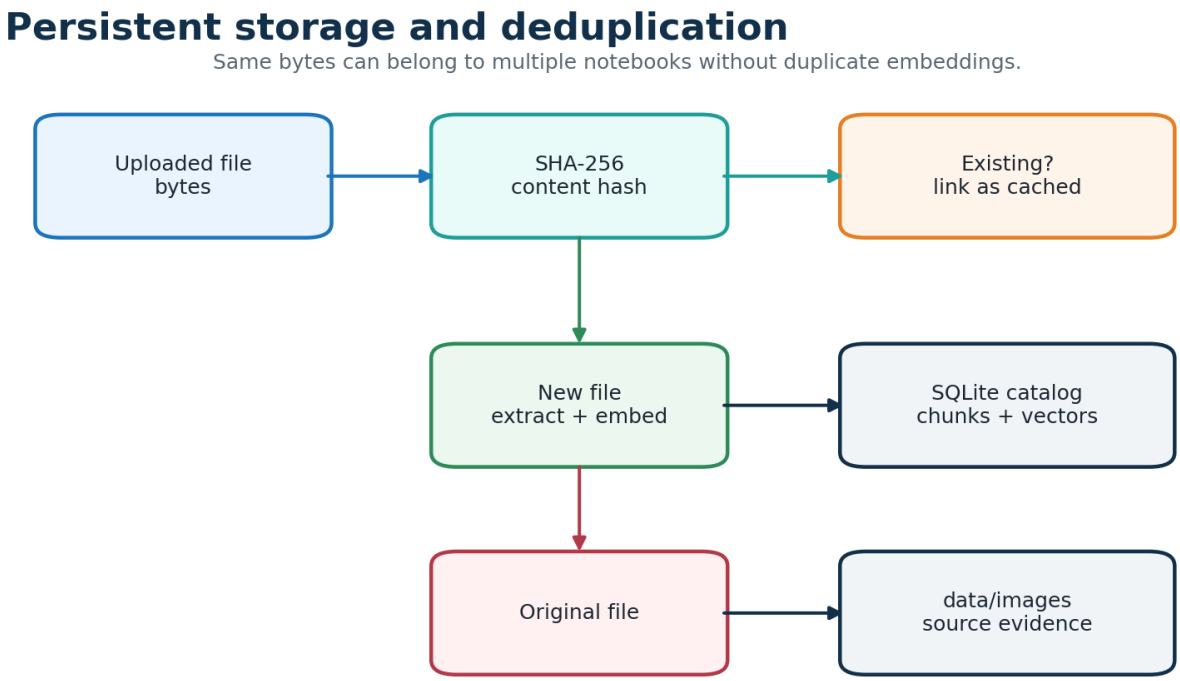



*Figure 7. SHA-256 controls one-time extraction and embedding work.*

Each upload is hashed before extraction. If the digest already exists, the service inserts a notebook membership link and returns status cached. It does not repeat OCR, chunking or embedding. If a file changes by even one byte, it receives a new digest and is indexed as a new version. This behavior is deterministic and avoids relying on filenames alone.

The persistent data directory contains knowledge.sqlite3, original documents and extracted images. Docker mounts it through rag_data; native deployment keeps it under the repository's data directory. Restarting services, rebuilding the application image or rebooting the host does not remove the catalog. The project explicitly warns against docker compose down -v because it deletes the volume.

| Scenario | Expected status | Embedding work |
|---|---|---|
| Same file, same bytes | cached | None |
| Same file, modified bytes | indexed | Once for new hash |
| Same file added to another notebook | cached/link | None |
| New supported file | indexed | Once |
| Unsupported extension | skipped | None |

# Knowledge graph design

*Current relationship model and proposed extension*

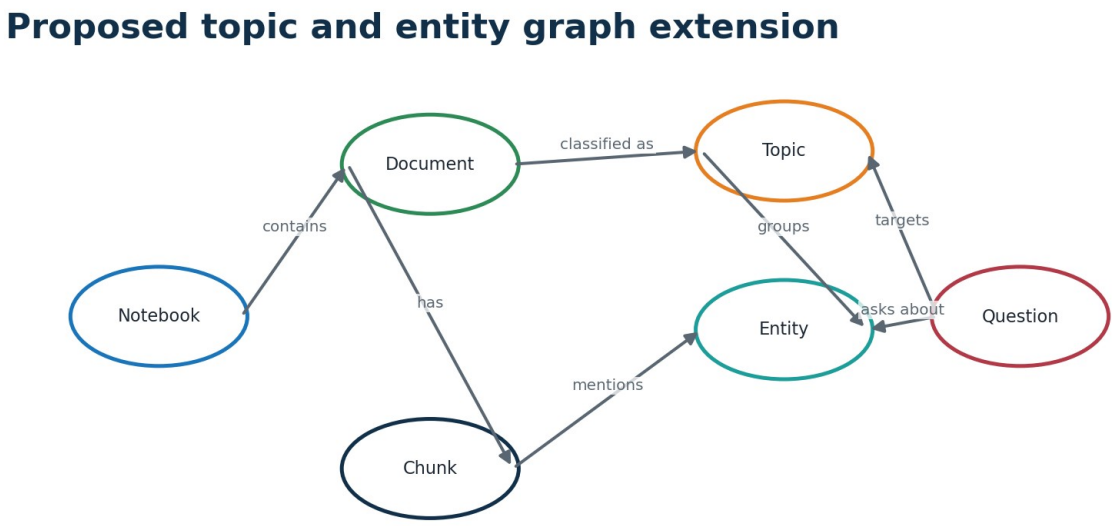



*Figure 8. Proposed topic/entity graph layered over the current index.*

The current code already stores useful graph-like relationships: notebooks contain documents; documents produce page- or sheet-aware chunks; chunks can be associated with extracted images. These links are sufficient for notebook scoping and source traceability, but they are not yet a standalone entity graph with nodes, typed relations or graph traversal.

A future graph layer should extract entities such as missions, spacecraft, sensors, locations, dates, organizations and technical parameters. It can create Topic, Entity, Relation and Mention tables in SQLite or a local Neo4j deployment. The graph should be built asynchronously after indexing so a slow extraction step does not block basic search.

| Graph element | Example | Retrieval use |
|---|---|---|
| Topic | Trajectory prediction | Topic filter and related-topic expansion |
| Entity | LEO, ISRO, orbit number | Exact entity matching |
| Relation | mission uses sensor | Multi-hop context |
| Mention | chunk 184 mentions LEO | Evidence link back to source |

Graph-aware ranking can add a small score bonus when a retrieved chunk is connected to the question's entities or topic. The dense-plus-FTS5 path should remain the fallback because it is already local, simple and fast.

# API, web interface and MCP

*Ways users and tools interact with the system*

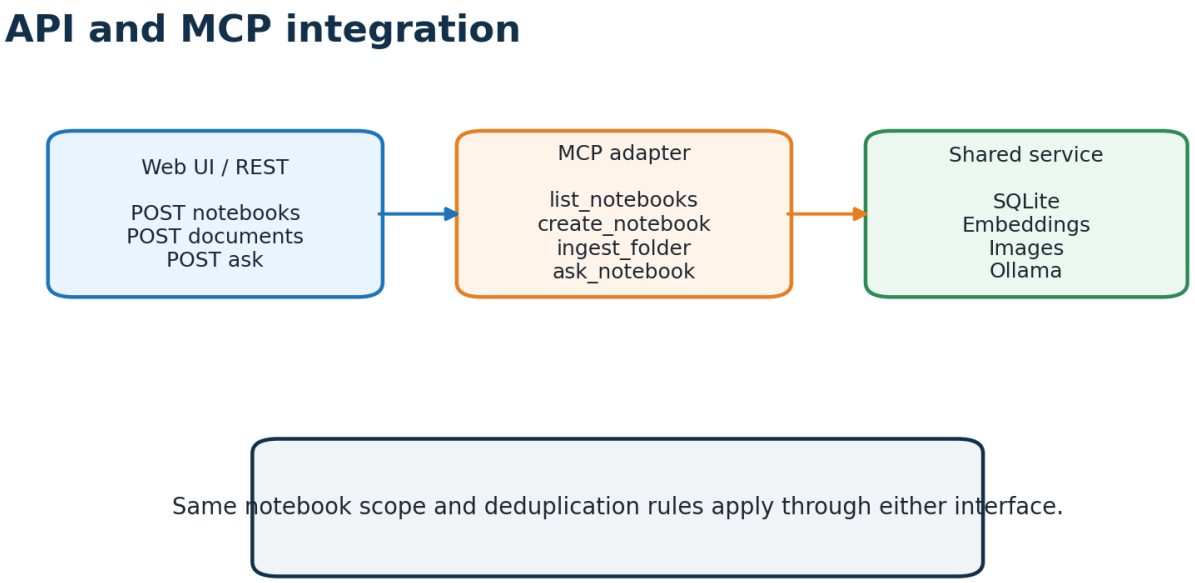



*Figure 9. REST and MCP share the same persistence and retrieval implementation.*

| Endpoint/tool | Purpose |
|---|---|
| GET /health | Service state and counts for notebooks, documents and chunks |
| GET /api/notebooks | List notebooks and document counts |
| POST /api/notebooks | Create a named notebook |
| POST /api/notebooks/{id}/documents | Upload files for indexing |
| POST /api/notebooks/{id}/ask | Retrieve, answer and return sources/images |
| MCP ingest_folder | Recursively index a local folder |
| MCP ask_notebook | Ask through a tool-enabled client |

The web UI is intentionally lightweight so it can be served by the API without a separate frontend build. It supports folder selection through the browser's directory input, displays indexing status and lists sources in the answer. Swagger at /docs is useful for integration testing and for demonstrating request/response shapes to a guide or deployment team.

# Deployment architecture

*Single PC first, lab server next*

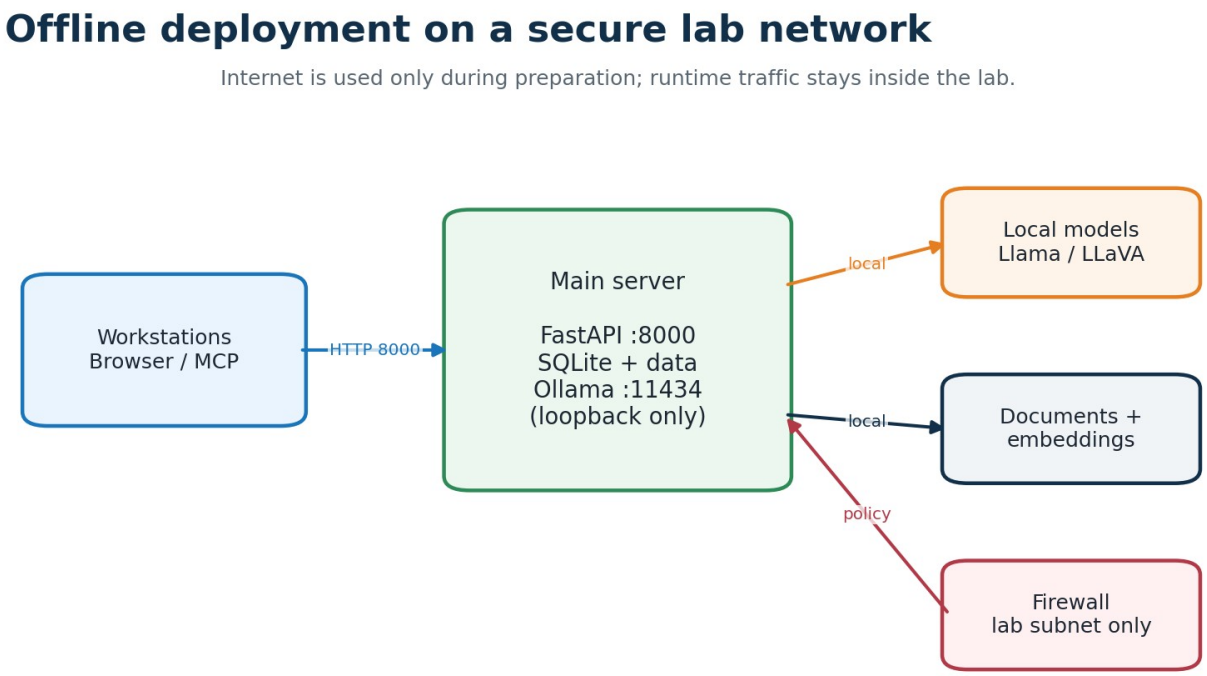



*Figure 10. Recommended trust boundary for a lab deployment.*

The simplest demonstration uses native Python and Ollama on one PC. This avoids Docker Desktop, WSL and Hyper-V, and is the recommended path when the host must be prepared quickly. The connected preparation machine downloads Python packages, embedding/OCR weights and Ollama models. The target machine receives the source, wheel bundle, model directories and optional data directory.

Docker Compose is also supported for Ubuntu and Windows Docker Desktop. The rag container serves port 8000, while Ollama is bound to 127.0.0.1:11434. Named volumes retain application data and model files. For a later lab rollout, bind only the API to the server interface, allow port 8000 from the approved subnet, and keep Ollama inaccessible to workstations.

| Deployment mode | Best use | Persistence |
|---|---|---|
| Native Python + Ollama | Fast demonstration; no Docker/WSL | data/ and models/ |
| Docker Compose | Repeatable Ubuntu/server deployment | rag_data and ollama_models volumes |
| Lab server | Multiple trusted workstations | API on subnet; Ollama loopback |

# Offline installation and operations

*Preparation once; runtime without internet*

A reproducible offline bundle should include the repository source, operating-system-specific wheel files, the Python installer, the Ollama installer, embedding and OCR model directories, and optionally the pre-existing data directory. A virtual environment must be rebuilt on the target operating system; copying a Windows environment to Ubuntu or vice versa is unsafe.

| Stage | Windows native example | Ubuntu native example |
|---|---|---|
| Prepare environment | .\scripts\setup_native.ps1 | ./scripts/setup_native.sh |
| Start Ollama | .\scripts\run_ollama.ps1 | ./scripts/run_ollama.sh |
| Pull models | ollama pull llama3.2:3b; ollama pull llava:7b | same commands |
| Start API | .\scripts\run_native.ps1 | ./scripts/run_native.sh |
| Verify | curl.exe http://127.0.0.1:8000/health | curl http://127.0.0.1:8000/health |

After verification, disconnect the target from the internet. Do not run pip install, ollama pull or Docker build on the isolated machine. Normal operation consists of starting Ollama, starting FastAPI, selecting a notebook, indexing new material and asking questions. Back up data/ or the Docker data volume while services are stopped.

The first indexing operation can be CPU-intensive because it performs extraction, OCR and embedding. Later questions benefit from the warm embedding cache and warm Ollama model. The README includes Windows, Ubuntu, Docker and no-WSL paths for the guide to reproduce the deployment.

# Security, privacy and operational controls

*Controls for a secure office*

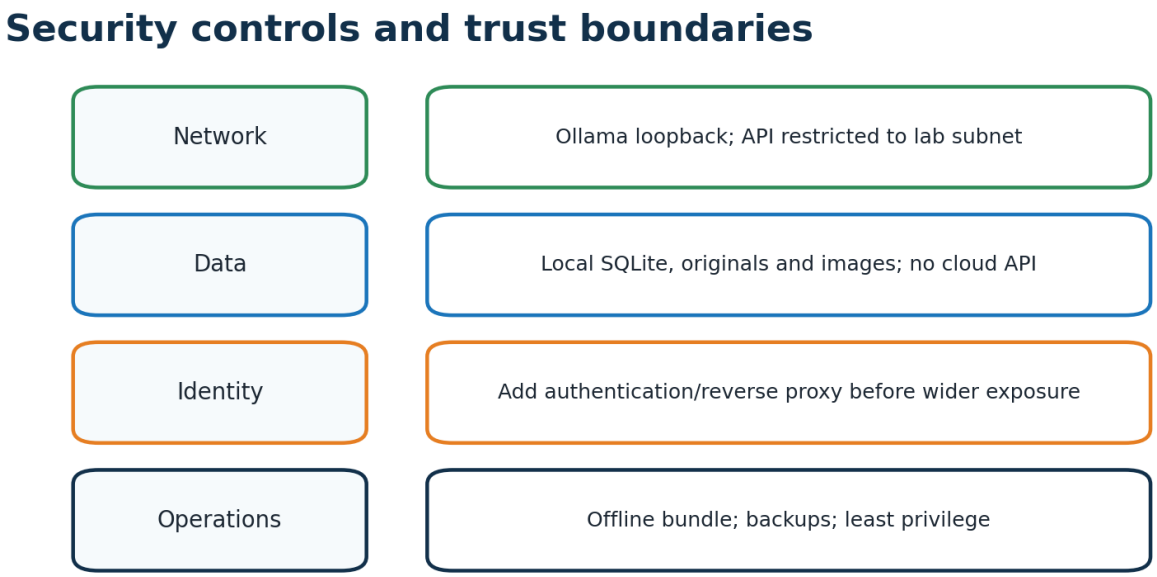



*Figure 11. Security controls mapped to the local trust boundary.*

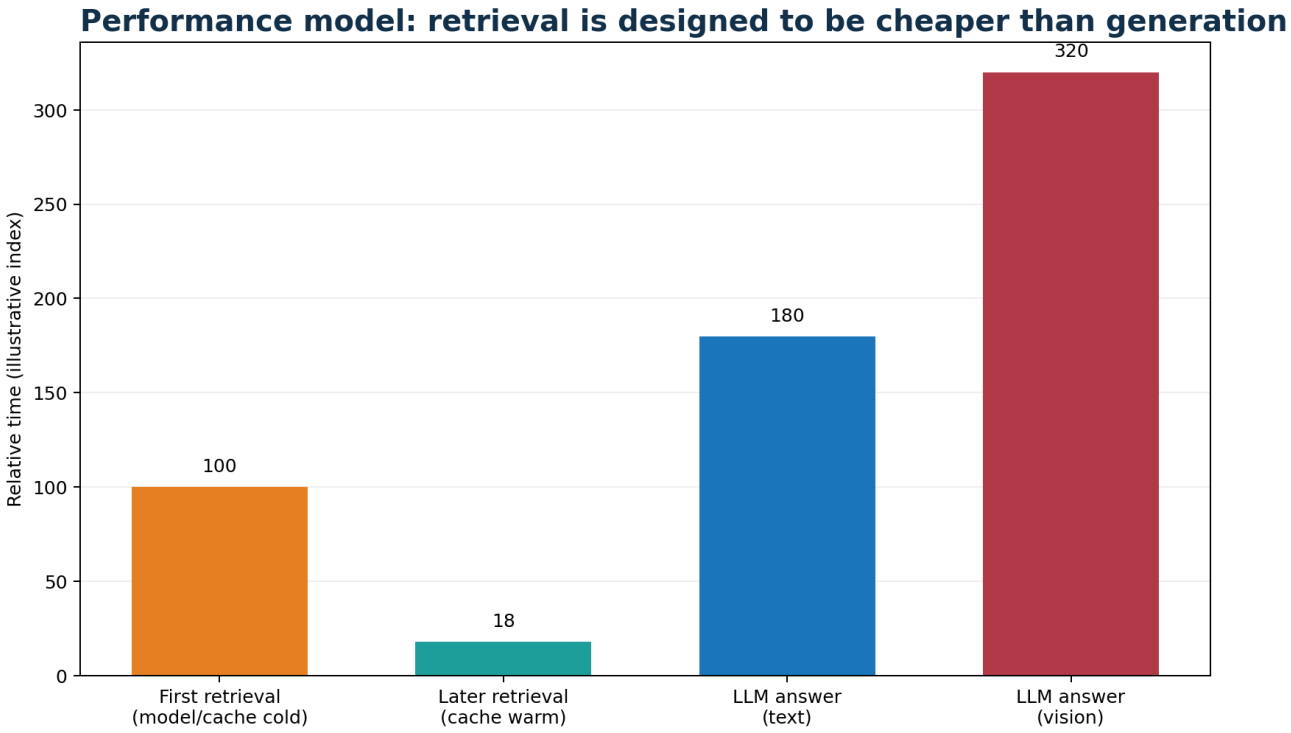
The project is designed to keep document content inside the deployment host. The runtime makes no cloud API call; the only model request is to the local Ollama service. The default API and Ollama bindings are loopback-only. When network access is required, the administrator should expose only port 8000 to a known lab subnet and keep 11434 private.

| Risk | Current mitigation | Required hardening before production |
|---|---|---|
| Unauthorized API access | Loopback default; subnet firewall guidance | User authentication, HTTPS, role policy |
| Sensitive backups | Local volumes and explicit backup guidance | Encrypted backups and access control |
| Prompt/source leakage | Local models and notebook scope | Audit logs and per-user authorization |
| Model supply chain | Offline model directories after preparation | Checksum and approved model registry |
| Resource exhaustion | Top-k/context limits | Upload size limits, quotas and monitoring |

The current system should be treated as a trusted-lab prototype until authentication, authorization, audit logging and formal backup controls are added. This boundary is stated explicitly so deployment decisions remain evidence-based.

# Performance and scalability

*How to make retrieval faster*




Illustrative engineering model, not a benchmark; measure on the target PC with its corpus and model quantization.

*Figure 12. Illustrative timing model; measure on the target hardware.*

The architecture places optimization effort where it has the most impact. Retrieval uses a cached in-memory matrix and FTS5, while generation is bounded by model context and token limits. The API reports retrieval and answer timings separately, allowing a proper demonstration benchmark rather than relying on subjective response speed.

| Measurement | Method | Success signal |
|---|---|---|
| Index throughput | Files/minute by format and size | Stable incremental ingest |
| Warm retrieval | Repeated questions after cache warm-up | Low retrieval_ms |
| Answer latency | Text vs vision model timing | Model choice matches hardware |
| Cache reuse | Repeat same upload | cached; no new chunks |
| Corpus growth | 1k/10k/100k chunks | Know when SQLite needs replacement |

For larger collections, the next performance steps are a persistent approximate-nearest-neighbor index, batch embedding during ingestion, background graph extraction, and a bounded per-notebook cache. SQLite remains appropriate for an initial single-server corpus because it keeps deployment simple and offline.

# Validation, limitations and future work

*What is verified and what remains*

| Area | Current verification | Remaining work |
|---|---|---|
| Code integrity | Python compilation succeeds for main.py and mcp_server.py | Automated CI test suite |
| Persistence | Schema, hashes, volumes and README workflow reviewed | Run full corpus test on target PC |
| Retrieval | Hybrid semantic/FTS5 implementation present | Collect latency baseline with real documents |
| Images | OCR extraction and source URLs implemented | Evaluate difficult diagrams and handwriting |
| Offline | Native and Docker instructions provided | Rehearse transfer bundle with no network |
| Knowledge graph | Notebook/document/chunk links available | Entity/topic/relation extraction |
| Security | Loopback defaults and firewall guidance | Authentication, HTTPS and audit trail |

The main limitation is that the current baseline is a document RAG service rather than a completed knowledge graph platform. It can be made graph-aware without discarding the existing index: extract entities from chunks, persist typed edges, and add graph features to candidate generation and ranking.

Recommended next milestones are: (1) implement a graph schema and entity extraction job; (2) add login and notebook permissions; (3) add a benchmark command that records indexing and retrieval timings; (4) introduce background workers for large folder imports; and (5) add encrypted, tested backup and restore procedures.

# Conclusion and guide discussion

*A practical foundation for secure offline document intelligence*

Offline Notebook Document QA provides a practical first deployment for a secure office: it accepts common files and folders, embeds them once, preserves evidence, retrieves quickly within a selected notebook, returns image sources, and uses local Ollama models for grounded answers. The same persistent data model supports a browser, REST clients and MCP tools.

The implementation is intentionally modular. The current SQLite and hybrid retrieval path is small enough to deploy on one PC and transparent enough to validate. A future knowledge graph can be layered on top of the same document, chunk and notebook identifiers, adding topic navigation and multi-hop retrieval without requiring cloud services.

| Decision for guide review | Recommended question |
|---|---|
| Demonstration hardware | Is the target PC sufficient for llama3.2:3b and optional llava:7b? |
| Corpus boundary | Which notebooks and file types should be tested first? |
| Security boundary | What subnet, authentication and backup policies apply? |
| Graph scope | Which entities and relationships matter to the lab domain? |
| Success metric | What retrieval latency and answer quality are acceptable? |

Repository and implementation reference: github.com/dhxvxn/URSC_RAG. The accompanying README contains step-by-step native Windows, native Ubuntu, Docker, offline bundle, lab network and troubleshooting instructions.

Prepared for technical review. The report describes the repository state inspected on 10 November 2026 and separates implemented behavior from proposed extensions.